

CSC360 Final Project: Insertion Sort and Binary Search Algorithm Implementation in RISC-V Assembly

Mark Stotzer*, Dawson Blakeslee**

Hamm School of Engineering at the University of Mary, Bismarck, ND

*mpstotzer1@umary.edu, **diblakeslee1@umary.edu

Abstract—This paper explores two fundamental algorithms—insertion sort and binary search—and their implementation in RISC-V 32-bit assembly. The algorithms will be examined in the C programming language, then translated to assembly. They will be analyzed by means of clock cycles, CPI, instruction count, execution time, and forwarding behavior to determine the effectiveness of the hand-generated assembly implementations compared to the compiler-generated versions. The paper will walk through how Ripes was used to evaluate each design, and hazards and forwarding related to each algorithm will also be discussed.

Index Terms—Cycles per instruction (CPI), execution time, pipelining, pointer, index, hazard

I. INTRODUCTION

SEARCHING and sorting data structures is fundamental to all data science, so fast algorithms are critical. Sorting places data in an order that is predictable, which correlates to improved performance of searching algorithms and other data analysis. Together, these algorithms are capable of many useful computing tasks that make up modern operating systems, search engines, and databases.

II. BACKGROUND AND SCOPE OF PROBLEM

All the algorithms and testing in this paper will be designed for a RISC-V 32-bit environment, comparing the single-stage and five-stage pipelining process on speed.

The insertion sort solves the problem of being able to sort an unsorted array in the simplest way possible. Sorting an array makes it easier to process, search, and analyze data, as many operations work more efficiently with sorted arrays. Insertion sort works by building the sorted array one element at a time, inserting an indexed value and comparing, and sorting into the correct position relative to previously sorted elements. It works best for small datasets, as stepping through each array element can be comprehensive and time consuming for large arrays.

The binary search implemented here will be fairly straightforward, but will be tailored to utilize a five-stage pipeline. It is the standard method, usually used for arrays, trees, and array-like structures, that performs searches in $O(\log n)$ time. It is so widely used that even small optimizations are well worth while. As such, refining the assembly code is a highly useful exercise. In this project, three designs will be examined:

a naive implementation, a compiler-optimized version, and a modified version that shares aspects from each.

III. DESIGN APPROACH

A. Insertion Sort

The design process for the insertion sort algorithm began with research into how the algorithm functioned, as to best understand the problem, solution, and implementation. Next, a high-level flowchart was developed from the methods found during the research stage, and can be found in Figure 1. From there, C code was made to accomplish the insertion sort task, which could then be more easily translated to RISC-V assembly. After the C implementation, a low-level flowchart (Figure 2) was created, which proved very useful in the proceeding step-by-step translation into assembly. After the translation to assembly, the program was tested in Ripes to verify completeness and to observe certain metrics provided by Ripes. The C program was then translated once again into assembly, but this time using an optimized 'gcc' style compiler. Finally, the optimized compiler program was compared to the hand-generated program in Ripes to document improvements in CPI, clock cycles, instruction count, and execution time.

B. Binary Search

For simple algorithms like binary search, the first step was drawing a high-level flowchart (see figure 3). Then, it was converted into C code and tested for bugs. This was converted into a low-level flowchart (figure 4) and then, given the simplicity of the algorithm, converted into assembly by hand to produce the naive implementation. An optimized compiler-generated algorithm, referred to as just the compiler algorithm, followed, and both were tested in Ripes. Finally, after studying the hazards, they were modified into one final algorithm, called the modified algorithm.

The algorithms were tested by averaging the cycles it took to find an element in the deepest level of the tree and two deepest level searches for items not in the tree (too low and too high, respectively). This produced the results seen in figure 7.

IV. FLOWCHARTS AND DESIGN PROGRESSION

A. Insertion Sort

The high-level flowchart shown in Figure 1 basically walks through the C implementation of the insertion sort, and the flowchart in Figure 2 goes through the assembly implementation. First, an array and the array size are taken as inputs, then an index is implemented that steps through the array until it reaches the array size. A different variable that represents the current value is then set to the current array value, and another index is created for the value left of the current value. The left index is decremented until it reaches zero or if an array element is found to be smaller than the current value. Elements larger than the current value are then stored in the correct location, and the process is repeated.

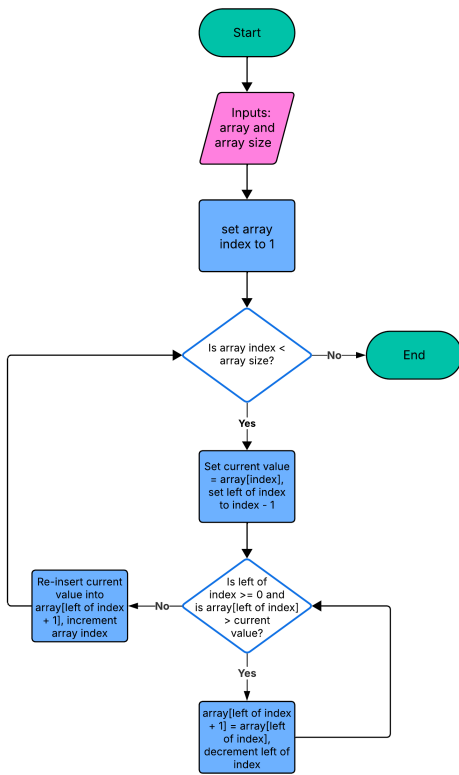


Fig. 1. High-Level Insertion Sort Flowchart

B. Binary Search

The binary search is simple, and is explained best visually (see fig 3). Essentially, you start with a sorted structure to be searched, usually an array or a tree, and set Left and Right pointers at the start and end respectively. Then, calculate the midpoint between them and check the midpoint value against the search value. If the search value is in the upper half of the array, the left pointer moves to the middle and the process repeats; if the search value is in the lower half of the array, the right pointer moves to the middle and the process repeats. Eventually, either the value is found or the left and right pointers hit each other and the value is not present.

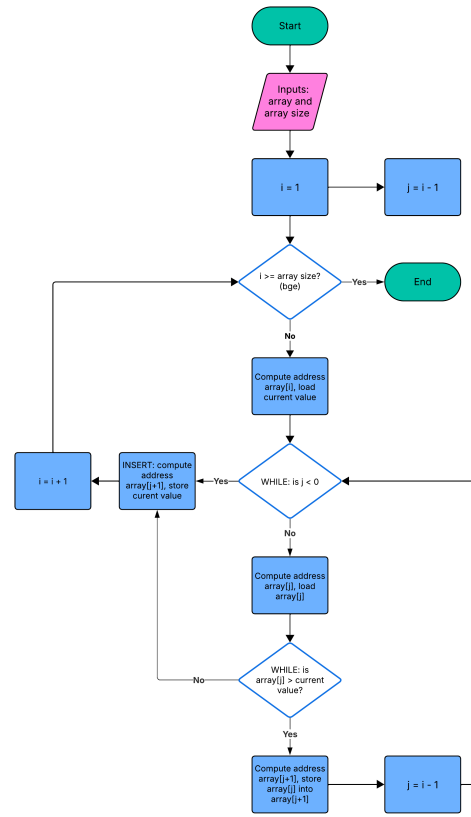


Fig. 2. Low-Level Insertion Sort Flowchart

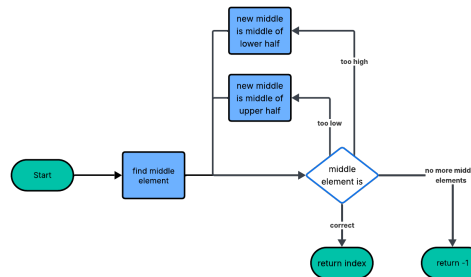


Fig. 3. High-Level Binary Search Flowchart

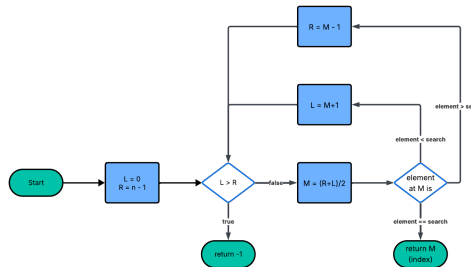


Fig. 4. Low-Level Binary Search Flowchart

V. PIPELINING AND HAZARDS

A. Insertion Sort

There are many hazards in the insertion sort implementation and they exist in three different categories: load-use hazards,

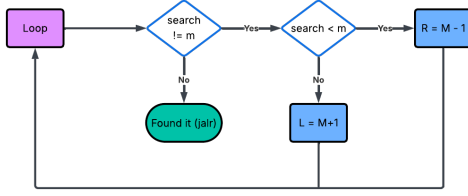


Fig. 5. Non-Pipelined Naive Branching

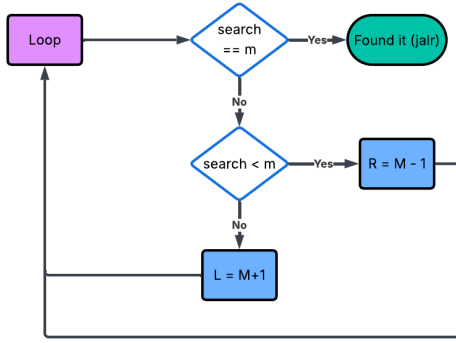


Fig. 6. Pipelined Modified Branching

data hazards, and control/branch hazards. The load-use hazard occurs in the *while* loop, when the array value of the left index is being loaded in one instruction, and then used to branch in the next instruction. This fails without pipelining due to the load instruction writing in the MEM/WB stage, while the branch instruction needs the value in the EX stage. The data hazards occur in each loop due to dependencies from *slli* and *add* instructions, which are most easily resolved by placing unrelated instructions between them. Finally, there are many branch hazards due to uncertainty of the variables in the *for* loop and *while* loop.

B. Binary Search

Looking at the different algorithm diagrams, there are two hazards. First is a load-use hazard "lw x31, 0, x31" followed by a branch statement using x31. This is common to all three since the code could not be rearranged to place this earlier.

The second hazard is a branch hazard rooted in the arrangement of branch statements in the assembly code. In the naive algorithm (Fig. 5), one branch statement jumped to the next upon success in a daisy chain pattern. This caused the two pipelined instructions after to be flushed upon failure. Since there were three cases, the chance of any branch failing was higher than those of it succeeding. There was an opportunity for three branch hazards per loop. Both the modified and compiler algorithms prevented this by placing all branch statements adjacent to each other in the code (see Fig. 6). As such, a failure would fall through to evaluating the next branch, causing at most one branch hazard per loop.

VI. ASSEMBLY IMPLEMENTATION

A. Insertion Sort

The first assembly version was generated directly from the low-level flowchart shown in Figure 2. It was hand-generated line by line, without much consideration of a pipelined structure, or hazards.

After identifying the hazards discussed in section V, a pipeline-aware assembly implementation was created that addressed as many of these hazards as possible. In each loop with data hazards, unrelated *addi* instructions were placed in between the *slli* and *add* instructions so as to give the processor time to return the correct dependent values. There was not much to be done concerning the branch/control hazards, though, and the load-use hazard was also unavoidable in this iteration.

The final implementation was generated by a *gcc*-based compiler. The compiler produced a further optimized assembly file, that used a pointer-based loop style instead of index-based in the first two versions. The compiler also utilized more registers, thus reducing the data and load-use hazards present in the first and second implementations. Finally, the compiler also more heavily reordered the instructions beyond that of the second assembly version. These changes correlated to increased efficiency discussed further in section VII.

B. Binary Search

There were three different assembly implementations: the naive, the compiler, and the modified. The naive and modified are entirely the same except for the order of branching (see section V for details). The compiler version has the same branch ordering trick as the modified, but included two few extra instructions on top. First, it set the return register to -1 by default, saving a cycle later if the item was not found but wasting a cycle if it was. Second, it immediately checked if the array length was greater than zero, saving time for an invalid array parameter but wasting time on valid ones.

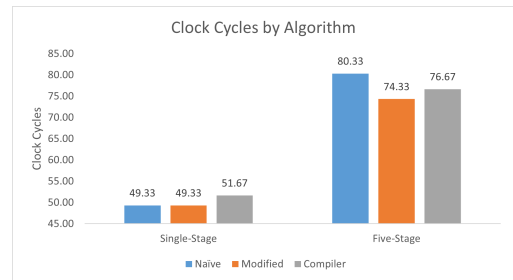


Fig. 7. Performance by Algorithm and Processor

VII. RESULTS

A. Insertion Sort

The insertion sort algorithm was evaluated using a 6 value array to be sorted. Each iteration of the algorithm was run through Ripes simulation software using both a single and five-stage processor. Data such as clock cycles, CPI,

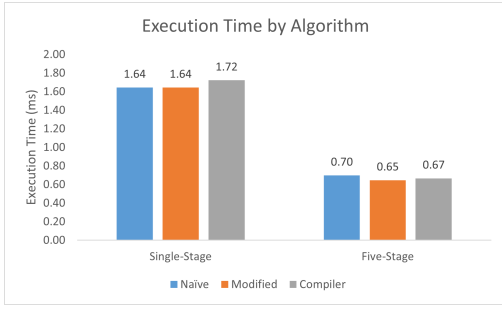


Fig. 8. Execution Time by Algorithm and Processor

instruction count, and clock rate were taken from Ripes to be evaluated. The latter three data points can be used to find execution time via the formula in equation 1.

$$\text{CPU Execution Time} = \frac{\text{Instr. Count} \cdot \text{CPI}}{\text{Clock Rate}} \quad (1)$$

The clock cycles observed from Ripes simulation software for each program are shown in Figure 9. The clock cycle data indicates no difference between the non-pipelined and pipelined versions of the assembly, and show that the compiler-generated code takes less clock cycles. However, the execution time data is even more interesting, and is presented in Figure 10. While the compiler code outperformed in clock cycles taken, it is clear that the pipelined implementation is the better performer in terms of 5-stage processor execution time, likely due to the higher clock rate achieved.

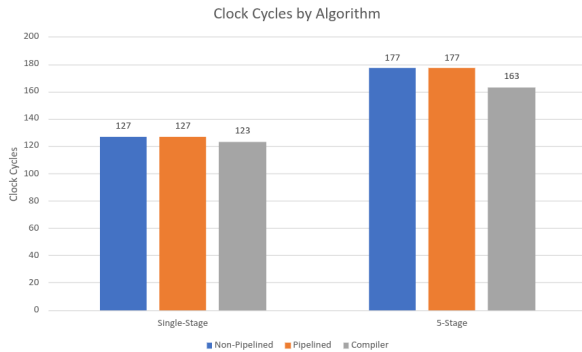


Fig. 9. Clock Cycle Comparison by Algorithm and Processor

B. Binary Search

The fastest algorithm, for both processors, was the modified algorithm. It tied with the naive in single-stage, given that there is no branch penalty, and beat the compiler by approximately two cycles. In the five-stage, the branch arrangement for modified and compiler easily beat out naive. Still, the compiler's extra introductory steps served to slow it down by 2 cycles.

It should be noted that single stage took half the cycles of five-stage, because the latter must flush and stall frequently. However, the advantage is that the five-stage runs five cycles in the time the single-stage runs one. In practice, due to added hardware complexity for pipelining, the clock period is only

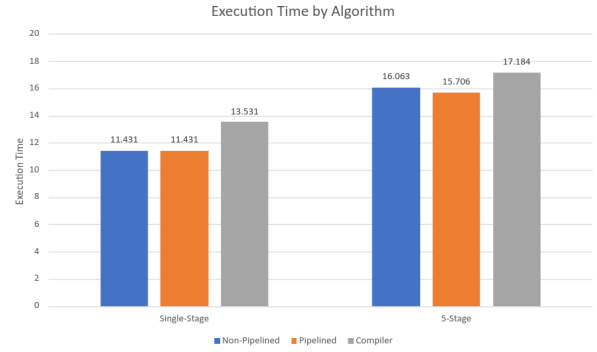


Fig. 10. Execution Time Comparison by Algorithm and Processor

around three times as fast. A standard clock period for RISC-V hardware was found online: 30 and 115 MHz for single and five-stage respectively. The estimated execution times, seen in figure 8, demonstrate the same pattern identified above: modified is fastest.

VIII. FURTHER RESEARCH

A. Insertion Sort

B. Binary Search

Branch hazards are the primary slowdown of the binary search algorithm, and although a branchless binary search is possible with clever loop unrolling [1] [2], this is beyond the scope of this project.

IX. CONCLUSION

The insertion sort... The fastest binary search algorithm was the modified version for two reasons. First, it cut out early-return checks, betting that the majority of data structures to be searched would be valid. More significantly, it reordered the branches to take full advantage of pipelining, minimizing branch delays.

REFERENCES

- [1] Muscar, A. (2023, February 18). Beautiful Binary Search in D. Alex's website. <https://muscar.eu/shar-binary-search-meta.html>
- [2] Skarupke, M. (2023, April 28). Beautiful branchless binary search. Probably Dance. <https://probablydance.com/2023/04/27/beautiful-branchless-binary-search/>